

· [KLDP.org](#) · [KLDP Wiki](#) · [KLDP.net](#) · [통합 검색](#) ·



[Subversion Book/Subversion ForCVS Users](#)

[🔍](#) [📄](#) [🔗](#) [🔄](#) [🔍](#) [📄](#) [🔗](#) [🔄](#) [XML](#)

[FrontPage](#) | [FindPage](#) | [TitleIndex](#) | [RecentChanges](#) |

[SubversionBook/BranchingAndMerging](#) › [SubversionBook/RepositoryAdministration](#)
› [SubversionBook/AdvancedTopics](#) › [SubversionBook/DeveloperInformation](#) ›
[SubversionBook/Reference](#) › [SubversionBook/SubversionForCVSUsers](#)

Login:

Password:

[Join](#)

Appendix A. CVS 유저를 위한 Subversion

Men seldom show dimples to girls who have pimples.

Table of Contents

- A.1.
- A.1. [리비전 번호의 의미가 다릅니다](#)
- A.2. [디렉토리의 버전](#)
- A.3. [더 많은 오프라인 작업](#)
- A.4. [상태와 갱신의 구별](#)
- A.5. [브랜치와 태그](#)
- A.6. [메타데이터의 속성](#)
- A.7. [충돌의 해소](#)
- A.8. [바이너리 파일과 변환](#)
- A.9. [버전 관리된 모듈](#)

[마을](#)
[FreeAsInFreedom](#)
[LinuxdocSgml/Tips-HOWTO](#)
[AutoInstallation](#)



kss@kldp.org

A.1.

이 부록은 Subversion에 익숙치 않은 CVS 사용자를 위한 가이드입니다. 이 장은 기본적으로 "10킬로 정도 떨어진" 두 개의 시스템간의 차이에 대한 일람입니다. 가능하면 해당되는 장으로 가는 참조 링크를 제공하도록 하겠습니다.

Subversion의 목표는 현재와 미래의 CVS 사용자를 이어받는 것이지만 CVS에서 "문제"가 되고 있는 행동을 개량하기 위해서 몇 가지 새로운 기능과 설계의 변경이 필요했습니다. 이것은 CVS 사용자에게는 지금까지의 습관을 고쳐야할 지도 모른다는 것을 의미합니다. 처음 CVS를 사용하기 시작할 때에는 이상했지만 지금은 익숙해져버린 것들입니다.

A.1. 리비전 번호의 의미가 다릅니다

CVS에서는 리비전 번호가 파일 마다 붙어 있었습니다. 그것은 CVS가 내부적으로 RCS를 이용하고 있었기 때문입니다. 각각의 파일에 대응하는 RCS 파일이 저장소에 있고, 그 저장소는 대충 프로젝트 트리의 구조와 일치하도록 배치됩니다.

Subversion에서 저장소는 하나의 파일 시스템과 같이 보입니다. 각각의 커밋은 완전히 새로운 파일 시스템 트리를 만듭니다. 본질적으로 저장소는 트리의 나열입니다. 리비전 번호는 각각의 트리에 붙여집니다. 누군가가 "리비전 54" 라고 말할 때 그들은 특정한 트리(그리고 간접적으로는 54번째의 커밋 후에 파일 시스템의 상태)에 대해 말하고 있습니다.

기술적으로는 "foo.c의 리비전 5" 라고 말하는 것은 옳지 않습니다. 대신에 "리비전 5에 있는 foo.c" 라고 말해야 합니다. 또한 파일의 변화에 대한 전제를 조심해 주세요. CVS에서는 리비전 5와 리비전 6의 foo.c는 항상 다릅니다. 그러나 Subversion에서는 아마 대부분의 경우 리비전 5와 6사이에서는 foo.c는 **바뀌지 않았을** 것입니다.

자세한 내용은>(을)를 보십시오.

A.2. 디렉토리의 버전

Subversion은 파일 내용만 추적하는 것이 아니라 트리의 구조의 변경을 추적합니다. 이것

은 CVS를 대체하기 위해서 Subversion을 새로 만든 큰 이유 중의 하나입니다.

CVS 사용자에게 이것이 다음을 의미합니다.

- **svn add** 와 **svn rm** 명령어는 디렉토리에 대해서도 동작합니다. **svn copy** 와 **svn move**도 그렇습니다. 그러나 이러한 명령어들은 저장소에 직접적인 변경을 가하지는 않습니다. 대신 작업 아이템은 단지, 추가 또는 삭제의 "예고"를 받을 뿐입니다. **svn commit** 명령이 실행될 때까지 저장소는 전혀 변경되지 않습니다.
- 디렉토리는 단순한 그릇이 아닙니다. 디렉토리도 파일과 마찬가지로 리비전 번호를 가지고 있습니다. (물론 "리비전 5 안에 있는 디렉토리 foo/ "라고 말하는 것이 더 적절합니다.)

두 번째 항목에 대해서 좀 더 설명합니다. 디렉토리의 버전 관리는 어려운 문제입니다. 우리는 여러 리비전이 섞인 작업본을 가능하도록 하고 싶기 때문에 이 모델을 남용하는 것에 대한 제한이 필요하게 됩니다.

이론적인 관점에서, 우리는 "디렉토리 foo의 리비전 5" 가 디렉토리 엔트리와 속성의 어느 특정의 모임을 의미한다고 정의합니다. foo에 파일을 추가하거나 삭제 하고 커밋했다고 합시다. 아직 우리가 foo의 리비전 5를 가지고 있다고 말하는 것은 거짓말입니다. 그러나, 커밋 후에 foo의 리비전 번호를 올렸다고 하면 이것도 역시 거짓말이 됩니다. 아직 작업본을 갱신하지는 않았기 때문에 저장소의 foo에는 다른 변경 사항들이 있을지도 모르기 때문입니다.

Subversion는 이 문제를 . svn 영역에 커밋된 추가와 삭제를 조용히 기록해두기만 합니다. **svn update**를 실행하면 저장소와 작업본 사이에서 모든 정보가 처리되고, 디렉토리의 새로운 리비전 번호가 올바르게 설정됩니다. **그러므로 작업본을 갱신한 뒤에야 디렉토리의 "완전한" 리비전을 가지고 있다고 안전하게 말할 수가 있습니다** 대부분의 경우 작업본은 "불완전한" 디렉토리 리비전을 포함하고 있습니다.

디렉토리의 속성을 바꾸고 커밋하려고 할 때도 비슷한 문제가 발생합니다. 보통 커밋은 작업 디렉토리의 로컬 리비전 번호를 올립니다. 그러나 역시 갱신하지 않았으므로 아직 저장소에서 받아오지 않은 변경 사항이 있을지도 모르기 때문에 불완전한 리비전입니다. 그래서 **최신 버전의 디렉토리가 아니면 디렉토리의 속성 변경을 커밋할 수 없습니다.**

디렉토리의 버전 관리의 제한에 대한 자세한 것은 >(을)를 보십시오.

A.3. 더 많은 오프라인 작업

최근에는 디스크 용량은 매우 싸고 풍부하게 되었지만 네트워크 대역폭은 그렇지 않습니다. 그 때문에 Subversion의 작업본은 이 귀중한 자원을 최적화하여 사용합니다.

. svn 관리 디렉토리는 CVS 디렉토리와 같은 목적을 위해서 사용되지만 한 가지 다른 점이 있습니다. 관리 영역에는 작업본의 "원본"이 읽기전용으로 보관되어 있습니다. 이것을 사용해 여러가지 작업을 오프라인으로 할 수 있습니다.

svn status

로컬에 일어난 모든 변경점을 표시합니다(>참조)

svn diff

변경점의 상세를 표시합니다(>참조)

svn revert

변경점을 취소합니다(>참조)

또한 캐쉬된 원본을 사용하여 커밋할 때 변경된 사항만을 전송합니다. CVS에는 없는 기능입니다.

마지막 항목은 새로운 기능입니다. 이 명령어는 로컬의 변경 사항을 취소하는 기능 뿐만 아니라 파일의 추가나 삭제 예정을 취소하는 기능도 가지고 있습니다. 파일의 변경을 취소하는데 이 명령을 추천합니다. **rm file; svn update** 같은 방법을 써도 되지만 이 방법은 갱신 명령의 목적을 흐립니다. 그리고 우리는 이 주제에 찬성하기는 합니다만...

A.4. 상태와 갱신의 구별

Subversion에서는 **cvs status** 와 **cvs update** 명령어 사이에 있는 많은 혼란을 없애려고 노력해 왔습니다.

cvs status 커맨드는 두 가지 목적이 있습니다. 첫 번째는 작업본의 로컬 변경 사항을 사용자에게 보여주는 것이고, 두 번째는 어느 파일이 최신 버전이 아닌지 사용자에게 보여주는 것입니다. 불행하게도 CVS의 매우 읽기 어려운 상태 표시 때문에 많은 CVS 유저는 이 명령의 장점을 전혀 살리고 있지 못합니다. 대신에 변경점을 빨리 보기 위해서 **cvs update** 또는 **cvs update -n** 명령을 실행하는 습관을 가지게 되었습니다. 이것에는 물론 부작용이 있어서 아직 다룰 준비를 하지 못한 저장소상의 변경과 머지를 일으켜 버립니다.

Subversion에서는 **svn status**의 출력을 사람도 쉽게 읽을 수 있고 프로그램으로 취급하기에도 간단하도록 개선했습니다. 또 **svn update**는 갱신된 파일에 대한 정보만 표시할 뿐 로컬의 변경은 **표시하지 않게** 했습니다.

svn status 명령어는 로컬에서 변경된 모든 파일을 보여줍니다. 기본적으로 이 명령은 저장소에 접근하지 않습니다. 이 명령이 꽤 많은 수의 옵션을 지원하기는 하지만 그중에서 다음 옵션들이 가장 자주 사용됩니다.

-u

저장소에 접근하여 최신 상황에 관한 정보를 보여줍니다.

-v

버전 관리하에 있는 **모든** 항목을 표시합니다.

-N

하위 디렉토리에 대한 재귀적 처리를 하지 않습니다.

status 명령어에는 두 가지 출력 형식이 있습니다. 디폴트인 "짧은 형식"에서는 로컬의 변경은 다음과 같이 표시됩니다.

```
% svn status
M      . /foo.c
M      . /bar/baz.c
```

--show-updates(-u) 옵션을 지정하면 좀 더 긴 출력 형식으로 보여줍니다.

```
% svn status -u
M      1047      . /foo.c
      *      1045      . /faces.html
      *      -      . /bloo.png
M      1050      . /bar/baz.c
Head revision: 1066
```

이 경우 세로줄 두 줄이 추가됩니다. 두 번째의 칼럼은 파일이나 디렉토리가 최신이 아닌 경우에는 asterisk('*')가 표시 됩니다. 3번째의 칼럼은 아이템의 작업 카피 리비전 번호입니다. 위의 예에서는, asterisk는 만약 갱신하려고 하면 faces.html 는 패치 되어 bloo.png 는 저장소(repository)에 신규 추가될 것이다 일을 나타내고 있습니다. (bloo.png의 다음에 오는 - 은 작업 카피에게는 아직 존재하고 있지 않는, 이라고 하는 것을 나타내고 있습니다)

마지막으로, 가장 자주(잘) 표시되는 스테이터스 코드의 간단한 통계를 태워 둡니다:

```
A      자원은 추가 예고되고 있습니다
D      자원은 삭제 예고되고 있습니다
M      자원은 로컬로 변경되고 있습니다
```

- C 자원은 충돌하고 있습니다(변경 개소가 저장소(repository)과 작업 카피와의 사이에 아직 완전하게는 머지 되고 있지 않습니다)
- X 자원은 이 작업 카피로부터 봐 외부의 것입니다(다른 저장소(repository)로부터 온 것입니다. >(을)를 참조해 주세요)
- ? 자원은 버전 관리하에는 없습니다.
- ! 자원은 없어졌는지, 불완전합니다(Subversion 이외의 다른 툴에 의해 삭제되었습니다)

Subversion 는 CVS의 **P** 와 **U** 코드를 연결해, 단지 **U** 를 표시합니다. 머지나 충돌이 일어날 때는, Subversion는 거기에 관한 모든 내용을 표시하는 대신에 단지 **G** 또는 **C** (을)를 표시합니다.

svn status에 관한 자세한 것은> (을)를 참조해 주세요.

A.5. 브랜치와 태그

Subversion은 파일 시스템 공간과 "브랜치" 의 공간을 구분하지 않습니다. 브랜치와 태그는 파일 시스템 내의 일반적인 디렉토리 입니다. 이것은 아마 CVS 유저가 극복하지 않으면 안 되는 제일 큰 심리적인 장해입니다. 이것에 대해서는>(브랜치와 머지) 전체를 읽어주세요.

A.6. 메타데이터의 속성

Subversion에서는 파일이나 디렉토리에 임의의 메타데이터(혹은 속성)을 지정할 수 있습니다. 속성은 작업본의 파일이나 디렉토리에 지정할 수 있는 임의의 이름/값 쌍의 나열이라고 생각할 수 있습니다.

속성명을 설정하거나 얻으려면 **svn propset** 와 **svn propget** 명령을 사용하십시오. 특정 개체의 모든 속성 목록을 보려면 **svn proplist**를 사용하면 됩니다.

보다 자세하게는>을 참조해 주세요.

A.7. 충돌의 해소

CVS는 파일 내용에 "충돌 표시"를 사용해 충돌을 알려주고, 갱신시에는 **C**를 표시합니다. 역사적으로는 이것은 문제를 일으켜 왔습니다. 그것은 CVS가 충분히 취급하지 않았기 때문입니다. 많은 사용자는 터미널에서 **C**가 휙 지나간 다음에 그냥 잊어버립니다. 충돌 표시가 파일에 들어있는 것을 잊어버리고는 그냥 커밋하는 일이 잦습니다.

Subversion 는 이 문제를, 충돌 표시를 좀 더 잘 다룰수 있도록 하여 해결하고 있습니다. Subversion 는 파일이 충돌 상태에 있다 일을 기억하고 있어서 **svn resolved**를 실행할 때까지는 커밋을 허락하지 않습니다. 자세한 것은 > 을 봐 주세요.

A.8. 바이너리 파일과 변환

대부분의 경우 Subversion은 바이너리 파일을 CVS 보다 좀 더 적절히 취급합니다. CVS는 RCS 를 이용하기 때문에 바이너리 파일이 변경되면 파일을 통채로 저장하는 수 밖에 없었습니다. 그러나 Subversion은 내부적으로 바이너리 차분 알고리즘을 사용하기 때문에 파일이 텍스트 파일인지 바이너리 파일인지 상관없이 차이점을 저장합니다. 이것은 모든 파일이 차분의 형태로(압축되어) 저장소에 저장된다는 것을 의미하고 또한 네트워크로 전송되는 데이터의 양은 얼마되지 않는다는 것을 의미합니다.

CVS 사용자는 -kb 플래그를 사용해서 바이너리 파일을 표시할 필요가 있었습니다. (키워드 확장이나 줄바꿈 변환 기능으로) 파일 내용이 깨지는 것을 막아야했기 때문입니다. 사람들은 때때로 이 작업을 잊어버렸습니다.

Subversion는 좀 더 편집중적인 방법을 취합니다. 우선 키워드 확장이나 줄바꿈 변환은 명시적으로 그러한 지시를 내리지 않으면 실행되지 않습니다 (자세한 것은> 과 >(svn:

kewyrds와 svn:eol-style부분) (을)를 봐 주세요). 기본적으로 Subversion은 모든 파일 데이터를 문자 그대로 단순한 바이트의 줄로서 취급하므로 파일은 항상 변환을 거치지 않고 저장소에 보존됩니다.

두 번째로 Subversion은 파일이 "텍스트"인지 "바이너리"인지 내부적으로 기록을 관리하지만 이 기록은 **작업본에만** 있습니다. **svn update**를 실행하면 변경된 텍스트 파일에 대해서는 문맥 머지를 하지만 바이너리 파일은 그런 작업을 수행하지 않습니다.

문맥 머지가 가능한지 어떤지를 결정하기 위해서 Subversion은 svn:mime-type속성을 조사합니다. 만약 파일이 svn:mime-type속성을 가지지 않거나 텍스트임을 표시한다면(예를 들어, text/*) Subversion는 그것을 텍스트 파일이라고 생각합니다. 그 이외의 경우, 파일은 바이너리라고 가정합니다. 또한 Subversion은 **svn import** 와 **svn add** 명령 실행시에 바이너리 검출 알고리즘을 실행하여 사용자를 돕습니다. 이 알고리즘은 쓸만한 추측을 해서 (가능하다면) 추가되는 바이너리 파일의 svn:mime-type 속성을 설정합니다. (만약 Subversion의 추측이 잘못되었다면 사용자가 언제든지 그 속성을 삭제하거나 수정할 수 있습니다.)

A.9. 버전 관리된 모듈

CVS와는 차이, Subversion의 작업 카피는 모듈로서 체크아웃 되었다 일을 기록하고 있습니다. 이것은 누군가가 모듈의 정의를 변경하고 나서 **svn update**를 호출했을 경우, 작업 카피를 적절히 갱신 하는 것을 의미합니다.

Subversion는 모듈을 있는 디렉토리 속성이 있는 디렉토리 의 모임으로서 정의합니다. > (을)를 봐 주세요.

[EditText](#) | [Refresh](#) | [FindPage](#) | [DeletePage](#) | [LikePages](#) | [Tour](#) | [Keywords](#) |



Last modified: 2004-12-21 13:58:59, Processing time: 0.0195 sec

[인터넷 익스플로러 7, 무료](#)

[웹 사진 앨범](#)

빠르고 안전한 브라우저. 지금 바로 Google에 [Picasa](#) 웹앨범으로 간단하게 온라인으로 사진 게시 및 공유하기